Checked vs Unchecked Exceptions

9 min read Updated 28 Jul 2026 Topic 27 of 65

Bookmark Mark done

Why it matters

This is a design question disguised as a language question. The mechanical answer — "checked must be declared, unchecked need not" — takes ten seconds. What interviewers actually want is your opinion on *when to use which*, because that reveals whether you have maintained a large codebase. It is also one of the few places where the wider Java community openly disagrees with the language designers, and knowing that debate is a genuine signal.

The concept, simply

Checked exceptions are part of a method's signature. The compiler enforces that every caller either catches them or declares them. The intent was: "this can fail for reasons outside your control, and a caller could plausibly recover."

Unchecked exceptions — `RuntimeException` and `Error` and their subclasses — need no declaration and propagate silently until something catches them. The intent was: "this is a bug, or the JVM is broken."

	Checked	Unchecked
Extends	`Exception` (not `RuntimeException`)	`RuntimeException` or `Erro`
Compiler	Must handle or declare	No requirement
Intent	Recoverable, external	Programming error
Examples	`IOException`, `SQLException`	`NullPointerException`, `Il`

	Checked	Unchecked
In lambdas	Painful — most functional interfaces forbid them	Fine

How it works

The rule is enforced at compile time and interacts with inheritance in a way worth knowing: **an override may throw fewer or narrower checked exceptions, never new or broader ones**. That is Liskov substitution again from Chapter 2 — code written against the parent must keep compiling with a child.

JAVA Copy

```
class Parent { void read() throws IOException { } }

class Child extends Parent {
    @Override void read() throws FileNotFoundException { }   // ✓ narrower
}
class Child2 extends Parent {
    @Override void read() { }                                // ✓ none at
}
// class Child3 extends Parent {
//     @Override void read() throws Exception { }            ✗ broader —
// }
```

The argument against checked exceptions is worth being able to state, because most modern Java has quietly taken this side. Three complaints:

1. **They do not scale.** A checked exception thrown deep in a call chain must be declared by every method between there and the handler, so a low-level change ripples through dozens of signatures.
2. **They encourage swallowing.** Faced with a forced `try/catch`, the path of least resistance is an empty block — which is strictly worse than the exception propagating.

wrapper.

The evidence is in the ecosystem: Spring wraps every `SQLException` in an unchecked `DataAccessException`, Hibernate throws unchecked, and no language designed after Java has adopted checked exceptions — including Kotlin, which runs on the same JVM and deliberately dropped them.

The argument for is real too: a checked exception is documentation the compiler enforces. `InterruptedException` is a good example — making it checked forces every author to at least consider cancellation, and the alternative would be silent breakage everywhere.

Code examples

The rippling problem:

JAVA Copy

```java
// One low-level method starts throwing a checked exception…
String readConfig() throws IOException { … }

// …and every caller up the chain must now declare or handle it.
Config parseConfig() throws IOException { return parse(readConfig()); }
Service buildService() throws IOException { return new Service(parseConfig
void startup() throws IOException { buildService().start(); }
// Four signatures changed because of one implementation detail nobody
// in the middle can do anything about.
```

The lambda problem, and the two honest answers:

JAVA Copy

```java
List<Path> paths = …;

// ✗ Does not compile: Function.apply cannot throw a checked exception.
// paths.stream().map(p -> Files.readString(p)).toList();
```

```
        return Files.readString(p);
    } catch (IOException e) {
        throw new UncheckedIOException(e);   // the JDK's own answer
    }
}).toList();

// ✓ Option 2 — a small helper, when this happens more than twice.
@FunctionalInterface
interface ThrowingFunction<T, R> { R apply(T t) throws Exception; }

static <T, R> Function<T, R> unchecked(ThrowingFunction<T, R> f) {
    return t -> {
        try { return f.apply(t); }
        catch (Exception e) { throw new RuntimeException(e); }
    };
}
paths.stream().map(unchecked(Files::readString)).toList();

// Note that the JDK shipping UncheckedIOException is a tacit admission th
// checked exceptions do not fit the functional style.
```

Designing your own, which is what a follow-up question usually becomes:

JAVA Copy

```
// ✓ Unchecked — the caller passed nonsense. This is a bug, not a conditic
public class InvalidOrderException extends RuntimeException {
    public InvalidOrderException(String message) { super(message); }
}

// ✓ Checked — the caller genuinely can and should react, e.g. by offering
//   the customer a different payment method.
public class PaymentDeclinedException extends Exception {
    private final String declineCode;
    public PaymentDeclinedException(String declineCode, Throwable cause) {
        super("payment declined: " + declineCode, cause);
        this.declineCode = declineCode;
    }
    public String declineCode() { return declineCode; }
```

```
// other than logging and giving up?" Yes → consider checked. No → uncheck
```

Under the hood

The JVM does not know about checked exceptions. It is purely a `javac` rule. The `throws` clause is recorded in the class file as an attribute for the compiler's benefit, but the JVM will happily let any exception propagate from anywhere. That is why **sneaky throws** works — a generics trick that throws a checked exception without declaring it, exploited by Lombok's `@SneakyThrows`:

JAVA Copy

```java
@SuppressWarnings("unchecked")
static <E extends Throwable> void sneakyThrow(Throwable e) throws E {
    throw (E) e;                // erasure means E is unchecked at run ti
}
// Callable without any throws clause. It works because the cast disappear
// at compile time and the JVM never verifies exception declarations.
// Legal, occasionally useful, and it makes callers unable to see what can
```

The practical modern consensus, which is a good closing line in an interview: default to unchecked for anything the caller cannot meaningfully act on, use checked sparingly for genuine recoverable branches in your own API, and never let a checked exception from a lower layer leak upward unwrapped — translate it at the boundary into something meaningful for that layer, always preserving the cause.

Common mistakes

- **Making every custom exception checked.** Most represent bugs or unrecoverable conditions and should be unchecked.
- `throws Exception` **on a signature.** Tells callers nothing and forces them to catch everything.

- **Catching a checked exception just to satisfy the compiler.** If you cannot handle it, wrap it with the cause and rethrow.
- **Letting `SQLException` reach the controller.** Translate at the boundary — that is what Spring's `DataAccessException` does for you.
- **Widening `throws` in an override.** Will not compile, and rightly so.
- **Wrapping without the cause.** The most expensive small mistake in the chapter.
- **Reaching for sneaky-throws to dodge design.** It hides what can fail from every caller.

Interview lens

Q: "Checked vs unchecked exceptions?"
A: Checked extend `Exception` but not `RuntimeException`, and the compiler forces every caller to catch or declare them — they were meant for recoverable conditions outside the program's control, like I/O failing. Unchecked are `RuntimeException` and `Error` subclasses, need no declaration, and signal programming errors or JVM failures. Only `javac` enforces the distinction; the JVM has no notion of it.

Q: "Which would you use for a new exception in your own code?"
A: Unchecked by default. The test I apply is whether a caller has a realistic action on catching it beyond logging and giving up. If not — bad argument, invalid state, a downstream service being unreachable — unchecked, because forcing a `try/catch` just produces empty blocks. If there genuinely is a branch, like a declined payment where the caller offers another method, a checked exception documents that and the compiler enforces it.

Q: "Are checked exceptions a good idea?"
A: The industry has largely voted no. They do not scale — one low-level change ripples through every signature in the chain; they encourage swallowing, because an empty catch is the easiest way to satisfy the compiler; and they do not work

reason, and Kotlin dropped them entirely. The counter-argument is that they are compiler-enforced documentation, and something like `InterruptedException` genuinely benefits from that.

Q: "How do you throw a checked exception from a lambda?"

A: You cannot directly — the standard functional interfaces do not declare them. Either catch inside the lambda and wrap in an unchecked exception, which is what the JDK's own `UncheckedIOException` exists for, or define a throwing functional interface and a small adapter that wraps for you. Whichever way, keep the original as the cause.

Q: "Can an override change the `throws` clause?"

A: It can narrow it or drop it entirely, but never broaden it — a subclass cannot declare a new or wider checked exception. Otherwise code written against the parent would suddenly fail to compile when handed a child, which is Liskov substitution being enforced by the language. Unchecked exceptions are unrestricted.

Q: "What is sneaky throws?"

A: A trick that throws a checked exception without declaring it, using generics and erasure so the cast disappears at compile time. It works because checked exceptions are a `javac` rule and the JVM never verifies them — Lombok's `@SneakyThrows` does exactly this. It is legal, and I would avoid it in shared code because callers can no longer see what a method can fail with.

Cheat-card

- Checked = extends `Exception`, not `RuntimeException` → **handle or declare**.
- Unchecked = `RuntimeException` / `Error` → no declaration needed.

- Only `javac` enforces it — the JVM has no idea (hence sneaky throws).
- Override may **narrow or drop** `throws`, never broaden.

- Default to **unchecked**; checked only where the caller has a real branch.
- Lambdas cannot throw checked → wrap (`UncheckedIOException`) or use a throwing-interface adapter.
- Translate at layer boundaries (Spring's `DataAccessException`) — **always keep the cause**.
- Never `throws Exception`. Never an empty catch.

QUICK CHECK — no marks, just for you.

1. Who enforces the checked-exception rule?

- ◯ The security manager
- ◯ Both javac and the bytecode verifier
- ◯ The JVM, at class-load time
- ◯ javac only — the JVM has no notion of checked exceptions, which is why the 'sneaky throws' trick works

Check answer

2. Can an overriding method declare a broader checked exception than the method it overrides?

- ◯ No — it may narrow or drop checked exceptions but never broaden them, or code compiled against the parent would stop working
- ◯ Yes, if the class is abstract
- ◯ Only for unchecked exceptions

Yes, as long as it is still an Exception subclass

Check answer

3. Why can a lambda passed to Stream.map not throw IOException?

- ◯ Because streams catch all exceptions internally
- ◯ Because Function.apply does not declare a checked exception, and none of the standard functional interfaces do
- ◯ Because IOException is unchecked in a lambda context
- ◯ Because lambdas cannot throw any exceptions

Check answer

‹ Previous
The Throwable Hierarchy

Next ›
try, catch, finally and Control Flow

LEARN MORE

ARTICLES & DOCS

Unchecked exceptions — the controversy
docs.oracle.com

Exceptions (JLS §11)
docs.oracle.com

VIDEOS



Difference between Checked and Unchecked Exception | Exception Handling in Java by Deepak

Java Interview Kit



Java Tutorial # 14 | Exception Handling in Java - Checked and Unchecked Exceptions

java9s

Hand-picked references for this lesson.